

Android exported activities security risks and give developer recommendations

Assumptions

- You own this Android test app, so pulling, decompiling, and static analysis are fully authorized.
- The app is currently installed on your device (or you have the APK file on your machine).
- You have adb (Android Debug Bridge), apktool, and jadx installed. If not, I'll include install hints.
- Focus is on insecure storage (plaintext SharedPreferences, world-readable files, unencrypted databases) and hardcoded endpoints (URLs, IP addresses, API keys, tokens in source code).
- All commands are static analysis – no dynamic execution or modification of the app.

Commands / Code

1. Pull the APK from your device (if needed)

bash

List installed packages to identify your test app

```
adb shell pm list packages | grep -i <yourappkeyword
```

Get the APK path for your package (replace com.example.testapp with actual package)

```
adb shell pm path com.example.testapp
```

Pull the APK to your machine (the path will be something like /data/app/~~base.apk)

```
adb pull /data/app/com.example.testapp-/base.apk testapp.apk
```

2. Quick strings overview (first look for URLs/keys)

bash

```
strings testapp.apk | grep -E "(https?://|api\.[endpoint|secret|password|token])" | sort -u
```

3. Decompile resources with apktool (AndroidManifest, resources, smali)

bash

Decode the APK into a directory (use -f to overwrite existing)

```
apktool d -f testapp.apk -o testappdecoded
```

View the AndroidManifest.xml (check for exported activities, permissions)

```
cat testappdecoded/AndroidManifest.xml | grep -E "exported|permission"
```

Search for hardcoded strings in resources (strings.xml, etc.)

```
grep -r "https\?:://" testappdecoded/res/ --include="*.xml"
```

4. Decompile to Java with jadx

bash

Decompile the APK to readable Java source

```
jadx -d testappjava testapp.apk
```

Or use the GUI for interactive browsing

```
jadx-gui testapp.apk
```

5. Search for insecure storage patterns in decompiled Java

bash

SharedPreferences insecure storage (plaintext, MODEWORLDREADABLE deprecated but still appears)
grep -r "getSharedPreferences\|openFileOutput\|MODEWORLDREADABLE\|MODEWORLDWRITEABLE"
testappjava/

SQLite open or create database

grep -r "openOrCreateDatabase\|SQLiteDatabase\|rawQuery\|execSQL" testappjava/

Internal files – check for FileOutputStream, Context.MODE

grep -r "FileOutputStream\|openFileOutput" testappjava/

Hardcoded cryptographic keys or passwords

grep -r "SecretKey\|KeySpec\|encrypt\|decrypt\|password\|passwd" testappjava/

Hardcoded endpoints/URLs

grep -r "https\?://\|http://localhost\|10\.\0\.\2\.\2\|api\.\|baseUrl\|BASEURL" testappjava/

Search for common API key patterns

grep -r "apikey\|apiKey\|ApiKey\|APIKEY" testappjava/

6. (Optional) Check assets and libs for additional static data

bash

Look for .json, .txt, .cfg files that may contain endpoints or credentials

find testappdecoded/assets -type f -exec strings {} \; | grep -E "(https?://|password|secret)"

What each command does

- adb shell pm list packages – Lists all installed apps; filter for yours.
- adb shell pm path – Gets the APK file path on the device.
- adb pull – Copies the APK to your computer.
- strings – Extracts readable text from binary; first quick pass for obvious URLs/keys.
- apktool d – Decodes APK into smali and resources; allows inspection of AndroidManifest and resource files.
- grep (various) – Searches decompiled source for insecure storage APIs, file modes, database operations, and hardcoded endpoints.
- jadx -d – Converts DEX/APK to readable Java source for deeper analysis.
- jadx-gui – GUI version for interactive browsing (e.g., double-click to jump to definitions).

Next safe steps

1. Run each command in order – start with strings to get a quick inventory of exposed data.
2. Review the most concerning results:
 - Any URL containing localhost, 10.0.2.2, or hardcoded IP addresses – likely development leftovers.
 - Any password, secret, or token in plaintext – insecure credential storage.
 - SharedPreferences using MODEWORLDREADABLE (now deprecated but still present in older apps) – other apps can read this.
3. Use jadx-gui to navigate to the exact lines where insecure storage calls or endpoints are used – read the surrounding code to understand context.
4. Compile a short report listing:
 - File name and line number of each insecure storage call.
 - Hardcoded endpoints and their purpose.
 - Severity (e.g., exposed credentials = critical, localhost URL = medium).

5. Fix the findings – move endpoints to a configuration file that can be injected at build time; store secrets in Android Keystore or encrypted SharedPreferences (EncryptedSharedPreferences).

Note: If you want to also check for exported activities (related to your previous topic), run `grep "exported=\"true\"" testappdecoded/AndroidManifest.xml` – that will flag components accessible by other apps. Combine this with the insecure storage findings for a complete risk picture.